

GPUMODE Lecture Study Notes: Lecture 11 – Sparsity

Core Problem the Lecture Addresses

This lecture addresses the practical GPU systems problem of **sparsity**: how to remove unnecessary neural network weights while actually obtaining runtime speedups on modern GPUs.

The mathematical idea is simple: if many weights are zero, then multiplying by those weights should require less work. However, the systems problem is much harder:

Multiplying by zero is only fast if the hardware and kernel implementation actually skip the multiply. If the computation still walks through the same dense loop structure, sparsity gives little or no speedup.

A neural network layer such as a linear layer computes:

$$Y = XW^T + b$$

where X is the activation matrix, W is the weight matrix, and b is the bias. If many entries of W are unnecessary, we would like to replace them with zeros:

$$W_{ij} \leftarrow 0$$

and then compute:

$$Y = XW_{\text{sparse}}^T + b$$

faster than the dense version. The lecture emphasizes that sparsity has two separate parts:

1. **Pruning**: deciding which weights to zero out.
2. **Sparse execution**: making the resulting sparse matrix multiplication faster on GPU hardware.

The central trade-off is:

$$\text{Accuracy Preservation} \longleftrightarrow \text{Hardware-Efficient Structure}$$

Highly flexible sparsity patterns preserve model accuracy better but are hard to accelerate. Highly structured sparsity patterns are easy to accelerate but can damage accuracy more severely.

Required Background

Dense Matrix Multiplication

Most deep learning layers reduce to matrix multiplication. For matrices:

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}$$

the dense GEMM operation is:

$$C = AB$$

with entries:

$$C_{ij} = \sum_{k=0}^{K-1} A_{ik} B_{kj}$$

The operation count is approximately:

$$\text{FLOPs} = 2MKN$$

because each multiply-add counts as two floating-point operations.

GPUs are extremely good at dense GEMM because the computation is regular, tiled, parallelizable, and maps efficiently onto tensor cores.

Why Sparsity Is Nontrivial on GPUs

A CPU may benefit from unstructured sparsity at very high sparsity levels because it can skip individual nonzero entries using formats such as coordinate lists, CSR, or CSC. GPUs, however, depend on regular parallel work distribution.

A GPU wants thousands of threads doing similar work. If nonzero elements appear randomly, then:

- memory accesses become irregular,

- load balancing becomes difficult,
- warp lanes may do unequal useful work,
- metadata overhead increases,
- sparse kernels may lose to highly optimized dense kernels.

Thus, sparsity must be designed around the GPU execution model.

Pruning by Magnitude

A simple pruning method is magnitude pruning. Given weights W_{ij} , remove the entries with the smallest absolute values:

$$\text{score}(W_{ij}) = |W_{ij}|$$

If a weight has very small magnitude, such as 0.001, the intuition is that it may contribute little to the final output. For a linear operation:

$$y_i = \sum_j W_{ij} x_j$$

if W_{ij} is tiny, then its contribution $W_{ij} x_j$ may be small compared to other terms.

This is only a heuristic. A small weight can still matter depending on the activation distribution, downstream layers, and loss landscape. But it is a common baseline because it is simple and often works surprisingly well.

Main Concepts

Sparsity as Removing Useless Compute

The lecture starts from the observation that dense neural networks often use more parameters than needed. If some parameters can be removed while preserving accuracy, inference and potentially training can become faster.

Let W be a dense weight matrix and M be a binary mask:

$$M_{ij} \in \{0, 1\}$$

Then pruning creates:

$$W_{\text{pruned}} = M \odot W$$

where $\odot$ is elementwise multiplication. If $M_{ij} = 0$, the corresponding weight is removed.

The sparsity level is:

$$s = \frac{\#\{(i, j) : W_{ij} = 0\}}{\#\{(i, j)\}}$$

A matrix with 90% sparsity has:

$$s = 0.9$$

meaning only 10% of entries are nonzero.

Unstructured Sparsity

In **unstructured sparsity**, there are no constraints on where zeros appear. Any weight can be zeroed independently.

Example:

$$W = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 0 & 0 \\ 2 & 0 & 4 \end{bmatrix}$$

The benefit is maximum flexibility. The pruning algorithm can remove whichever weights seem least important.

The problem is that the nonzero entries may be scattered randomly. This makes GPU execution inefficient because the hardware cannot easily form regular tiles of useful work.

Sparse Coordinate Representation

One sparse storage format is coordinate format. Instead of storing all entries, store only:

- the nonzero values,
- their row and column indices.

For example:

$$W = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

can be stored as:

$$\text{values} = [1, 2, 3]$$

$$\text{indices} = [(0, 0), (0, 1), (0, 2)]$$

This saves memory when the matrix is sparse enough, but it introduces metadata overhead. The sparse representation is only better if:

$$\text{Sparse Compute Cost} + \text{Metadata Cost} < \text{Dense Compute Cost}$$

On GPUs, dense GEMM is so optimized that sparse formats must overcome a very strong baseline.

Structured Sparsity

In **structured sparsity**, entire rows, columns, filters, blocks, or layers are removed.

Examples include:

Remove row i

Remove column j

Remove an entire convolution filter

This is easy to accelerate because the resulting computation can often be represented as a smaller dense computation.

For example, if an entire column of W is removed, then the corresponding activation dimension can be skipped entirely.

However, structured sparsity can hurt accuracy because it removes large coherent pieces of the model.

Semi-Structured Sparsity: 2:4 Sparsity

Semi-structured sparsity is an intermediate point between unstructured and structured sparsity.

The lecture focuses heavily on NVIDIA's **2:4 sparsity**. In every group of four elements, exactly two are kept and two are zeroed.

For each group:

$$[w_0, w_1, w_2, w_3]$$

2:4 sparsity requires:

$$\#\{w_i \neq 0\} = 2$$

This gives a fixed sparsity level:

$$s = \frac{2}{4} = 0.5$$

So the matrix is always 50% sparse.

The advantage is that this pattern is regular enough for hardware acceleration while still allowing some flexibility over which two values are kept inside each group of four.

Block Sparsity

In block sparsity, zeros appear in blocks rather than individual elements. For example, a matrix may be divided into 32×32 blocks:

$$W = \begin{bmatrix} B_{00} & B_{01} & \cdots \\ B_{10} & B_{11} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

where each block B_{ij} is either dense or entirely zero.

The sparsity level is:

$$s_{\text{block}} = \frac{\#\{\text{zero blocks}\}}{\#\{\text{total blocks}\}}$$

Block sparsity is more hardware-friendly than unstructured sparsity because each surviving block can be computed as a dense tile. However, it is less flexible than unstructured sparsity, so recovering accuracy can be harder.

Hardware-Level Explanation

Why Dense GEMM Is So Fast

Dense GEMM maps well to GPUs because:

- computation is regular,
- memory accesses can be coalesced,
- tiles can be reused from shared memory,

- tensor cores can perform matrix multiply-accumulate efficiently,
- every warp can execute similar instructions.

A typical tiled GEMM loads tiles of A and B :

$$A_{\text{tile}} \in \mathbb{R}^{M_t \times K_t}$$

$$B_{\text{tile}} \in \mathbb{R}^{K_t \times N_t}$$

and accumulates:

$$C_{\text{tile}} += A_{\text{tile}} B_{\text{tile}}$$

This creates high data reuse. A value loaded from global memory can participate in many multiply-adds.

Why Unstructured Sparse GEMM Is Hard

Unstructured sparsity breaks the regularity that GPUs rely on. If every row has a different number and location of nonzeros, then different threads and warps may perform different amounts of work.

This can lead to:

- warp divergence,
- irregular memory loads,
- non-coalesced accesses,
- metadata reads,
- poor tensor core utilization.

Even if the mathematical FLOP count is lower, the real GPU runtime may not improve.

Metadata Overhead

Sparse formats need metadata to describe where the nonzero values came from.

For 2:4 sparsity, the nonzero values are packed together, but the system must store which two of every four elements were kept.

For every group of four elements, there are:

$$\binom{4}{2} = 6$$

possible ways to choose two nonzero locations. In principle, this requires at least:

$$\lceil \log_2 6 \rceil = 3$$

bits of information per group, though hardware-specific formats may encode metadata differently.

The compressed sparse representation consists of:

compressed tensor + metadata

Runtime speedup is limited by:

$$T_{\text{sparse}} = T_{\text{compressed loads}} + T_{\text{metadata loads}} + T_{\text{sparse MMA}} + T_{\text{epilogue}}$$

Even if the sparse multiply itself is theoretically $2\times$ cheaper, metadata and kernel overhead reduce the observed speedup.

Why 2:4 Sparsity Does Not Always Give 2x Speedup

Since 2:4 sparsity removes half the weights, the theoretical maximum speedup is approximately:

$$\text{Max Speedup} \approx 2\times$$

But the lecture mentions practical speedups closer to about $1.6\times$ for kernels in favorable cases.

Reasons include:

- metadata must be loaded and decoded,
- not all matrix shapes are equally favorable,
- sparse tensor core paths have constraints,
- epilogue operations may dominate,
- transpose or layout conversions may add overhead,
- fusion limitations can reduce end-to-end gains.

The GEMM dimensions matter. For:

$$C_{M \times N} = A_{M \times K} B_{K \times N}$$

speedup depends on M , N , and K . A large K dimension can expose more multiply-accumulate work to skip, while small or awkward dimensions may not saturate the sparse tensor core path.

cuSPARSELt Versus CUTLASS

The lecture discusses two NVIDIA-related backends:

- **cuSPARSELt**: NVIDIA library for sparse linear algebra, especially semi-structured sparsity.
- **CUTLASS**: template-based CUDA library for GEMM-like kernels.

cuSPARSELt provides optimized sparse kernels and useful features such as fused transpose support, but it is more of a black box. This makes custom fusion harder.

CUTLASS is more flexible because developers can write or modify kernels, but missing features must be implemented manually.

The systems trade-off is:

Black-box performance library versus customizable kernel framework

Mathematical Reasoning

Dense Versus Sparse Operation Count

For dense GEMM:

$$C = AB$$

with:

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}$$

the dense FLOP count is:

$$\text{FLOPs}_{\text{dense}} = 2MKN$$

If B has sparsity s , then the ideal sparse FLOP count is:

$$\text{FLOPs}_{\text{sparse, ideal}} = 2MKN(1 - s)$$

For 2:4 sparsity:

$$s = 0.5$$

so:

$$\text{FLOPs}_{\text{sparse, ideal}} = MKN$$

The ideal speedup is:

$$\text{Speedup}_{\text{ideal}} = \frac{2MKN}{MKN} = 2$$

But the actual speedup is:

$$\text{Speedup}_{\text{actual}} = \frac{T_{\text{dense}}}{T_{\text{sparse}}}$$

and T_{sparse} includes metadata, layout, compression, and fusion overhead.

Sparse Representation Cost

Let a dense matrix have n total elements. If the sparsity level is s , then the number of nonzeros is:

$$\text{nnz} = n(1 - s)$$

Dense storage cost is:

$$\text{Storage}_{\text{dense}} = n \cdot b_v$$

where b_v is bytes per value.

Sparse storage cost is approximately:

$$\text{Storage}_{\text{sparse}} = \text{nnz} \cdot b_v + \text{nnz} \cdot b_i$$

where b_i is the metadata/index cost per nonzero.

Sparse storage wins if:

$$n(1 - s)(b_v + b_i) < nb_v$$

Canceling n :

$$(1 - s)(b_v + b_i) < b_v$$

Solving for s :

$$s > 1 - \frac{b_v}{b_v + b_i}$$

This explains why unstructured sparsity often needs very high sparsity levels to be worthwhile.

Magnitude Pruning Rule

For a group G of weights, magnitude pruning keeps the largest entries by absolute value.

For 2:4 sparsity, each group contains four weights:

$$G = [w_0, w_1, w_2, w_3]$$

The mask is:

$$m_i = \begin{cases} 1, & |w_i| \text{ is among the top 2 magnitudes in } G \\ 0, & \text{otherwise} \end{cases}$$

The pruned group is:

$$G_{\text{pruned}} = [m_0 w_0, m_1 w_1, m_2 w_2, m_3 w_3]$$

Example:

$$G = [0.10, -2.00, 0.03, 1.20]$$

The two largest magnitudes are:

$$|-2.00| = 2.00, \quad |1.20| = 1.20$$

so:

$$M = [0, 1, 0, 1]$$

and:

$$G_{\text{pruned}} = [0, -2.00, 0, 1.20]$$

Accuracy Recovery

After pruning, the model output changes:

$$f(x; W) \neq f(x; W_{\text{pruned}})$$

The goal is to find a mask M such that:

$$f(x; W) \approx f(x; M \odot W)$$

for the target data distribution.

Accuracy can be recovered using:

- retraining,
- fine-tuning,
- calibration-style one-shot pruning,
- gradual sparsity schedules,
- sparse-aware training.

For large language models, retraining is expensive, so methods such as SparseGPT-style one-shot calibration are attractive.

Code Walkthrough

Representative Magnitude Pruning for 2:4 Sparsity

```
import torch

def prune_2_to_4(weight: torch.Tensor) -> torch.Tensor:
    # Assume the last dimension is divisible by 4.
    original_shape = weight.shape

    # Group consecutive elements into groups of 4.
    grouped = weight.reshape(-1, 4)

    # Compute magnitude scores.
    scores = grouped.abs()

    # Find the two largest magnitude values in each group.
    _, keep_indices = torch.topk(scores, k=2, dim=1)

    # Build a binary mask.
    mask = torch.zeros_like(grouped)
```

```
mask.scatter_(dim=1, index=keep_indices, value=1.0)

# Apply mask and restore original shape.
pruned = grouped * mask
return pruned.reshape(original_shape)
```

This code implements the conceptual pruning rule. Each group of four values keeps two values and zeros two values.

Hardware implications:

- The result satisfies the 2:4 sparsity constraint.
- The tensor is not automatically faster just because it contains zeros.
- To get speedup, the tensor must be converted into a compressed semi-structured representation and executed by a sparse kernel.

PyTorch Semi-Structured Sparse Conversion

The lecture describes a PyTorch API similar to:

```
import torch
from torch.sparse import to_sparse_semi_structured

linear = torch.nn.Linear(4096, 4096, bias=False).cuda().half()

with torch.no_grad():
    linear.weight.copy_(prune_2_to_4(linear.weight))

linear.weight = torch.nn.Parameter(
    to_sparse_semi_structured(linear.weight)
)
```

Conceptually, this does two separate things:

1. The pruning function inserts zeros in a valid 2:4 pattern.
2. `to_sparse_semi_structured` compresses the tensor and stores metadata.

The important lesson is:

Zeros in a dense tensor are not enough. The kernel must know that the tensor is sparse and must use a sparse execution path.

Linear Layer Layout Issue

PyTorch linear layers conventionally compute:

$$Y = XW^T + b$$

where X has shape:

$$X \in \mathbb{R}^{B \times K}$$

and W has shape:

$$W \in \mathbb{R}^{N \times K}$$

Sparse matrix multiplication implementations may only support the first matrix being sparse. But in PyTorch's linear layer, the sparse candidate is usually W , which appears as the second operand after transposition.

Using transpose identities:

$$XW^T = (WX^T)^T$$

This allows rewriting the operation so that W appears as the first matrix. However, explicit transposes can cost memory movement and runtime unless fused.

Fusion with torch.compile

The lecture emphasizes that `torch.compile` can help fuse surrounding operations.

For example:

$$Y = \text{ReLU}(XW^T)$$

may involve:

$$\text{sparse matmul} \rightarrow \text{transpose} \rightarrow \text{ReLU}$$

If the transpose and ReLU can be fused, the overhead is reduced. But if the sparse matmul call is a black-box library call, fusion into the sparse kernel itself may not be possible.

This matters especially when combining sparsity with quantization.

Combining Sparsity and Quantization

A simplified dynamic quantization flow is:

$$X_{\text{fp}} \rightarrow X_{\text{int8}}$$

$$W_{\text{fp}} \rightarrow W_{\text{int8}}$$

$$Y_{\text{int32}} = X_{\text{int8}} W_{\text{int8}}^T$$

$$Y_{\text{fp}} = \text{dequantize}(Y_{\text{int32}})$$

When sparsity is added, the weight path becomes:

$$W_{\text{fp}} \rightarrow W_{\text{pruned}} \rightarrow W_{\text{compressed sparse}}$$

The combined graph may look like:

$$\text{quantize} \rightarrow \text{sparse int matmul} \rightarrow \text{dequantize}$$

The performance challenge is that quantization and dequantization operations need to be fused into the sparse matmul epilogue. If they cannot be fused, the overhead can erase the expected speedup.

Performance Intuition

The Fundamental Performance Equation

A sparse method is useful only if:

$$T_{\text{sparse end-to-end}} < T_{\text{dense end-to-end}}$$

where:

$$T_{\text{sparse end-to-end}} = T_{\text{pruning/compression}} + T_{\text{sparse GEMM}} + T_{\text{metadata}} + T_{\text{layout}} + T_{\text{epilogue}}$$

For inference, pruning and compression can often happen offline:

$$T_{\text{pruning/compression}} \approx 0$$

during runtime. For training, pruning may happen repeatedly, so it becomes part of the runtime cost.

Inference Versus Training

For inference:

- weights are fixed,
- pruning can happen offline,
- compressed sparse weights can be stored ahead of time,
- runtime only pays sparse matmul and metadata costs.

For training:

- weights change every optimizer step,
- sparsification may need to happen repeatedly,
- both W and W^T may be needed,
- compressed transposes are difficult,
- memory savings may disappear or become a memory penalty.

The lecture notes that for semi-structured sparsity during training, one may need to keep both:

$$W_{\text{compressed}}$$

and:

$$W_{\text{compressed}}^T$$

because transposing a compressed sparse representation is not as simple as changing strides.

For dense tensors, transpose can be represented by changing strides:

$$\text{transpose}(W) \Rightarrow \text{same storage, different strides}$$

For compressed sparse tensors, the structure is encoded in metadata, so transpose requires reorganizing the compressed representation.

Why Sparse Training Is Mostly a Compute Play

During inference, sparsity can save both compute and memory. During training, semi-structured sparsity may not save memory because of extra compressed copies and masks.

Thus, sparse training is primarily about reducing matmul compute time:

$$T_{\text{sparsify}} + T_{\text{sparse matmul}} < T_{\text{dense matmul}}$$

If fast sparsification kernels exist, sparse training can produce end-to-end speedups.

Activation Sparsity

The lecture briefly mentions applying sparsity to activations. Activations after ReLU are naturally sparse because:

$$\text{ReLU}(x) = \max(0, x)$$

so all negative values become zero.

If activations are sparse, sparse computation may be possible on the activation matrix instead of the weight matrix.

This can matter because different matrix shapes produce different performance. Depending on the GEMM layout, making X sparse may be more beneficial than making W sparse.

Why LLMs Are Harder Than Vision Models

The lecture suggests that vision models may tolerate some sparsity surprisingly well, even zero-shot. For example, vision transformer or SAM-like models may retain much of their quality after 2:4 pruning.

LLMs are more fragile. Zero-shot pruning can severely degrade sentence classification or language modeling quality.

Possible reasons include:

- token prediction is highly sensitive to small logit changes,
- transformer MLP and attention layers may encode distributed information,
- errors compound autoregressively,
- language tasks may rely on precise rare-feature representations.

This means sparsity research must be evaluated by model family, task, and deployment setting.

Common Misconceptions

Misconception: Zeros Automatically Make Computation Faster

Zeros in a dense tensor do not automatically reduce runtime. A dense GEMM kernel still performs the same loop nest:

$$C_{ij} = \sum_k A_{ik} B_{kj}$$

even if many B_{kj} values are zero.

To get speedup, the system must use a sparse representation and a sparse kernel.

Misconception: 50% Sparsity Means 2x End-to-End Speedup

2:4 sparsity removes half the values, but practical speedup is lower because of metadata, shape constraints, memory movement, and fusion limitations.

The realistic equation is:

$$\text{Speedup} = \frac{T_{\text{dense}}}{T_{\text{sparse kernel}} + T_{\text{overheads}}}$$

Misconception: Unstructured Sparsity Is Always Best

Unstructured sparsity is best for accuracy flexibility but often poor for GPU performance. GPUs need regularity.

Misconception: Structured Sparsity Is Always Safe

Removing full rows, columns, filters, or layers is easy to accelerate but can remove too much information and hurt accuracy.

Misconception: Quantization and Sparsity Speedups Automatically Compose

If quantization gives speedup and sparsity gives speedup, their combination does not automatically give the product of both speedups.

The combined graph may introduce unfused operations:

$$\text{quantize} \rightarrow \text{sparse matmul} \rightarrow \text{dequantize}$$

If these are separate kernels, launch overhead and memory traffic can dominate.

Practical Takeaways

- Sparsity has two distinct problems: choosing zeros and exploiting zeros.
- GPU-friendly sparsity requires structure.
- 2:4 sparsity is attractive because it is hardware-supported and gives fixed 50% sparsity.
- Block sparsity can achieve higher sparsity levels, but accuracy recovery is harder.
- Sparse kernels must beat highly optimized dense GEMM, which is difficult.
- Metadata overhead is central to sparse performance.
- Inference benefits more naturally from offline compression.
- Training with sparsity is possible, but compressed transpose and repeated sparsification complicate the system.
- Combining sparsity with quantization requires careful fusion.
- CUTLASS offers customization; cuSPARSELt offers optimized black-box sparse routines.

Project or Experiment Ideas

Experiment 1: Dense Versus 2:4 Sparse Linear Layer

Implement a PyTorch benchmark comparing:

$$Y = XW^T$$

using:

- dense FP16 weights,
- 2:4 pruned dense weights,
- compressed semi-structured sparse weights.

Measure:

$$T_{\text{dense}}, \quad T_{\text{sparse}}$$

and compute:

$$\text{Speedup} = \frac{T_{\text{dense}}}{T_{\text{sparse}}}$$

Vary M , N , and K to see how matrix shape affects sparse speedup.

Experiment 2: Magnitude Pruning Accuracy Loss

Take a pretrained MLP or small vision transformer and apply magnitude pruning.

Compare:

- dense baseline accuracy,
- zero-shot pruned accuracy,
- fine-tuned pruned accuracy.

Measure the recovery gap:

$$\Delta_{\text{recovery}} = \text{Accuracy}_{\text{fine-tuned sparse}} - \text{Accuracy}_{\text{zero-shot sparse}}$$

Experiment 3: Quantization Plus Sparsity

Benchmark four variants:

1. dense BF16,
2. sparse BF16,
3. dense INT8 dynamic quantization,
4. sparse INT8 dynamic quantization.

The goal is to verify whether speedups compose:

$$\text{Speedup}_{\text{sparse+quant}} \stackrel{?}{\approx} \text{Speedup}_{\text{sparse}} \cdot \text{Speedup}_{\text{quant}}$$

Expect that fusion limitations may prevent perfect composition.

Experiment 4: Block Sparsity Simulation

Divide a weight matrix into blocks:

$$B_{ij} \in \mathbb{R}^{b \times b}$$

Compute each block norm:

$$\|B_{ij}\|_F = \sqrt{\sum_{p,q} B_{ij,pq}^2}$$

Remove blocks with the smallest norms.

Compare different block sizes:

$$4 \times 4, \quad 16 \times 16, \quad 32 \times 32$$

Study the trade-off between accuracy and speed.

Experiment 5: Nsight Compute Sparse Kernel Profiling

Profile dense and sparse GEMMs with Nsight Compute.

Look for:

- achieved occupancy,
- DRAM throughput,
- tensor core utilization,
- L2 cache behavior,
- instruction mix,
- memory coalescing,
- kernel launch overhead.

The goal is to understand whether sparse execution is compute-bound, memory-bound, metadata-bound, or launch-overhead-bound.

Visual Concept

Draw three matrices side by side. The first is dense with all cells filled. The second is unstructured sparse with random zeros scattered throughout. The third is 2:4 sparse, where every group of four cells has exactly two filled and two empty. Under each matrix, write: “accurate but hard to accelerate,” “flexible but irregular,” and “hardware-friendly compromise.”

Final Mental Model

Sparsity is not simply about having zeros. It is about creating zeros in a pattern that the GPU can exploit.

The full pipeline is:

$$W \rightarrow M \odot W \rightarrow \text{compressed sparse representation} \rightarrow \text{sparse GPU kernel} \rightarrow \text{speedup}$$

The performance question is:

Did we skip enough useful work to beat dense GEMM?

The accuracy question is:

Did we put the zeros in places the model can survive?

The core mental model is:

Sparsity is a contract between the model and the hardware. The model agrees to arrange zeros in a recoverable pattern; the hardware agrees to skip those zeros efficiently. If either side fails, sparsity does not pay off.